❸ The same database configured for R2DBC but using the JDBC driver

As you probably noticed, defining domain objects and adding a persistence layer in a reactive application is similar to what you would do with imperative applications. The main differences you encountered in this session are using the R2DBC driver instead of JDBC and having a separate Flyway configuration (at least until R2DBC support is added to the Flyway project: https://github.com/flyway/flyway/issues/2502).

In the next section you'll learn how to use `Mono` and `Flux` in your business logic.

8.2.3 Implementing the business logic with reactive streams

The Spring reactive stack makes it straightforward to build asynchronous, non-blocking applications. In the previous section, we used Spring Data R2DBC and didn't have to deal with any underlying reactive concerns. That's generally true for all the reactive modules in Spring. As a developer, you can rely on a familiar, simple, and productive approach for building reactive applications while the framework takes care of all the heavy lifting.

By default, Spring WebFlux assumes that everything is reactive. This assumption means that you're expected to interact with the framework by exchanging `Publisher<T>` objects like `Mono<T>` and `Flux<T>`. For example, the `OrderRepository` we created earlier will give access to orders as `Mono<Order>` and `Flux<Order>` objects instead of returning `Optional<Order>` and `Collection<Order>` like it would do in a non-reactive context. Let's see that in action.